Monitor de calibração de pluviômetros e medidor de vazão

O pedido de revisão e o que mudou no software

Sistema	Bancada de calibração: datalogger Campbell CR1000Xe e servidor de monitoramento (Python, Flask, SQLite)
Pedido	Revisão completa do sistema e aquisição da vazão por pulsos no CR1000Xe (texto integral no Anexo A)
Programa do datalogger em uso	Pluviometria_v3_com_vazao_4-20mA.CR1X
Revisão e data	Rev. 1, 28/10/2026
Elaboração	Renan (Elus Instrumentação) e Claude, assistente de IA da Anthropic
Destinatário	Responsável técnico pela bancada
Situação	Software implementado e verificado em emulador. Falta a verificação no equipamento (seção 3.2)

Resumo

O pedido foi revisar o sistema que adquire e grava as medições da bancada de calibração e acrescentar a leitura da vazão do SITRANS FM MAG 5000 pela saída de pulsos, sem perder o 4–20 mA que já funcionava: escolha entre analógico e pulsos no card de Vazão, fator K configurável no Método Comparativo, interface redesenhada e testes mostrando que nada do que existia quebrou. A seção 1 resume o pedido; o texto integral está no Anexo A.

No software, corrigimos dois defeitos críticos que não estavam documentados: a releitura indevida das tabelas do datalogger, que regravou parte do histórico sem temperatura e sem vazão, e duas cópias do servidor rodando ao mesmo tempo. Implementamos o método por pulsos com confirmação pelo dado do datalogger, escrevemos o programa v4 do CR1000Xe, refizemos a interface e otimizamos o servidor. Tudo passou em 221 testes automatizados e 105 verificações de ponta a ponta contra um emulador. Falta verificar no equipamento o programa v4, a ligação dos pulsos e o fator K (seção 3.2).

Sumário

1 O pedido . . . 3

1.1 Quem pediu e como . . . 3

1.2 O que foi pedido . . . 3

1.3 Restrições do pedido . . . 3

2 O que mudamos no software . . . 4

2.1 Leitura do datalogger (datalogger.py) . . . 4

2.2 Servidor (app.py) . . . 5

2.3 Medidor de vazão (calc.py, app.py, trials.py) . . . 6

2.4 Programa do datalogger (CRBasic v4) . . . 7

2.5 Interface (templates/, static/) . . . 7

2.6 Testes . . . 8

2.7 Desempenho . . . 9

3 Atendimento ao pedido . . . 9

3.1 Item a item . . . 9

3.2 O que depende da bancada . . . 11

Referências . . . 11

Apêndice A – Arquivos alterados . . . 12

Anexo A – Pedido original (texto integral) . . . 13

1 O pedido

1.1 Quem pediu e como

O pedido veio do responsável técnico pela bancada, como um roteiro de trabalho completo (texto integral no Anexo A). Ele parte do sistema como estava (balança MK30K1, termopar tipo T, pluviômetro de báscula e o MAG 5000 já lido em 4–20 mA) e pede uma revisão de engenharia de todo o caminho dos dados, do programa do datalogger até a tela, em seis etapas: inventário, diagnóstico, plano, implementação, verificação e entrega.

1.2 O que foi pedido

A Tabela 1 resume o pedido por assunto. Os números entre parênteses são as seções do texto original.

Tabela 1 – O pedido, por assunto

Assunto	O que foi pedido
Aquisição por pulsos (2)	Ler a vazão também pela saída de pulsos do MAG 5000, sem perder o 4–20 mA.
Seletor Analógico/Pulso (2.1)	Controle compacto no card de Vazão que mostre o método realmente ativo, diferencie o escolhido do confirmado e impeça trocas durante um ensaio.
Modo analógico (2.2)	Manter a lógica do 4–20 mA, com atenção aos limites de 4 e 20 mA, sem extrapolar.
Modo por pulsos (2.3)	Definir as variáveis do CRBasic e a janela de contagem; não tratar contagem como frequência; cuidar de vazões baixas e frequências altas.
Fator K (3)	Configurar no Método Comparativo, com unidade, validação e persistência; não inventar o K do MAG 5000.
Programa CRBasic (4)	Revisar o programa inteiro; não duplicar a contagem; não escrever no datalogger sem analisar; não presumir a ligação elétrica.
Servidor Python (5)	Revisar a leitura, os nomes dos campos (Flow_mode), o modelo de dados do medidor e os estados de erro.
Interface (6)	Redesenho profissional nas cores da Elus, com painel contextual, transições discretas e acessibilidade.
Desempenho (7)	Explicar os ciclos de recuperação e reduzir latência e processamento sem sobrecarregar o datalogger.
Integridade metrológica (8)	Não alterar cálculos sem justificativa e separar implementação, verificação e validação.
Testes (9)	Analógico, pulsos, troca de método e regressão, com dados sintéticos isolados dos reais.
Entrega (10)	Pacote com arquivos alterados, instalação, atualização do CR1000Xe, testes, pendências e reversão.

Fonte: pedido do responsável técnico (Anexo A).

1.3 Restrições do pedido

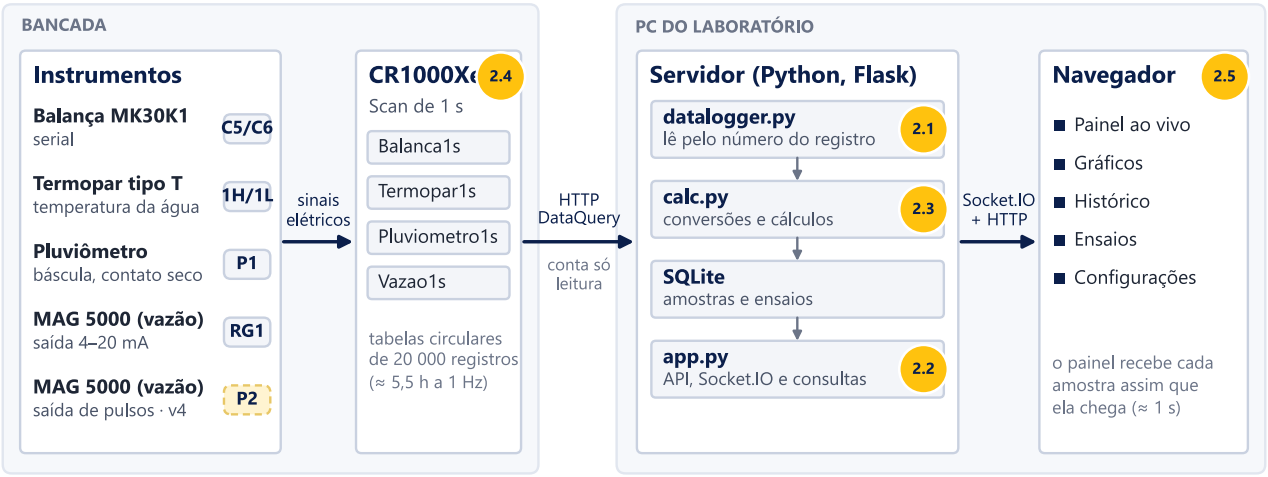
O pedido termina com uma lista de restrições (seção 11 do texto). As que mais pesaram nas nossas decisões:

- não inventar dados do medidor, o fator K nem a pinagem, e não presumir que a saída de pulsos já esteja configurada;
- não remover o 4–20 mA nem alterar os cálculos metrológicos sem justificativa;
- não habilitar escrita no datalogger sem analisar comunicação, segurança e persistência;
- não esconder erro de aquisição mostrando zero;
- não declarar testado o que não foi executado nem dizer que o sistema está validado só porque o código funciona.

2 O que mudamos no software

A Figura 1 mostra as partes do sistema. Os números em amarelo indicam a subseção que descreve cada alteração.

Figura 1 – Arquitetura do sistema e onde entrou cada alteração



Fonte: código-fonte do servidor e programa CRBasic.

2.1 Leitura do datalogger (datalogger.py)

Foi aqui que encontramos o defeito mais grave. O servidor pedia a cada tabela o registro seguinte ao último recebido. Quando o pedido chegava antes de o datalogger gravar esse registro, a API respondia a partir do registro mais antigo da tabela; o servidor entendia que a tabela tinha reiniciado, relia milhares de registros e regravava o histórico sem temperatura e sem vazão. Pelo log, isso aconteceu 16 vezes entre 22 e 28/10, e 92,7 % das 27 160 amostras do banco ficaram sem temperatura. Os ensaios encerrados não foram afetados, porque cada um guarda uma cópia imutável das suas amostras.

Agora o servidor pede a partir do último registro já recebido (o registro âncora), confere o número e o horário dele e o descarta (Figura 2). Nunca pede um registro que ainda não existe, e só aceita um reinício real quando a âncora some ou muda de horário. Mudamos também a gravação: regravar um horário que já existe agora só completa campos ausentes, nunca substitui uma leitura válida.

Deixamos pronta, mas desligada, a leitura das 4 tabelas em paralelo (DATALOGGER_PARALLEL=1). Ela mantém o número de consultas e pode reduzir a latência se o CR1000Xe atender pedidos simultâneos; só deve ser ligada depois de medida com o datalogger real.

Figura 2 – Consulta antes e depois da correção



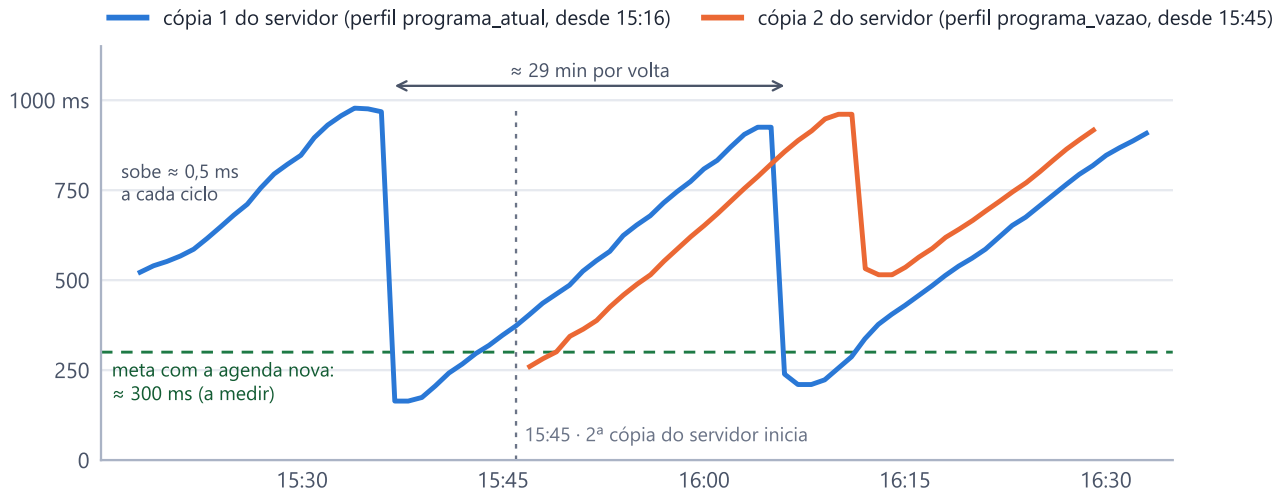
Fonte: datalogger.py, versões anterior e atual.

2.2 Servidor (app.py)

Em 27/10, das 15:45 às 16:33, o log mostra duas cópias do servidor rodando juntas, consultando o datalogger e gravando o mesmo banco. No Windows, o servidor web abre a porta com SO_REUSEADDR, e a segunda cópia sobe na mesma porta sem dar erro. Colocamos uma trava de instância única: uma segunda cópia registra o motivo no log e encerra.

Refizemos também a agenda das consultas. Cada espera era contada a partir do início do próprio ciclo, e o pequeno excesso se acumulava: a consulta atrasava cerca de 0,5 ms a cada ciclo, e a latência subia de 0,16 s a 1,1 s, voltando ao mínimo a cada ~29 min (Figura 3). Cada vez que a fase cruzava o instante de gravação do datalogger, o servidor pedia um registro ainda não gravado, que era justamente o gatilho do defeito da seção 2.1. A agenda nova usa horário absoluto, com a fase logo depois da gravação do Scan e nova tentativa em 120 ms. A meta é ~0,3 s, sem aumentar o número de consultas.

Figura 3 – Latência entre o carimbo do datalogger e a chegada ao servidor, antes da correção



Fonte: resumos por minuto de logs/servidor.log, 27/10/2026 (mediana móvel de 5 min).

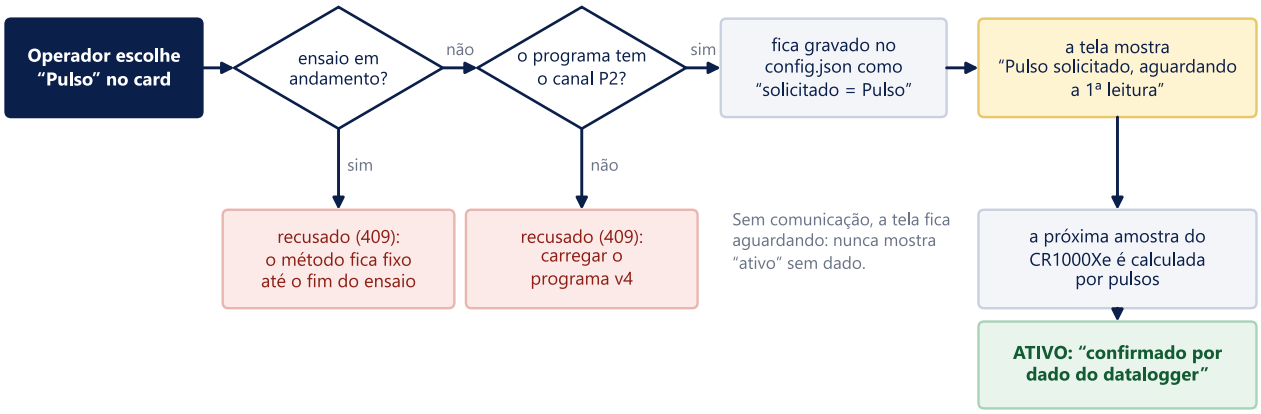
Outras mudanças no servidor:

- API do medidor: /api/flow (estado) e /api/flow/method (troca do método, recusada durante ensaio ou quando o programa não tem o canal);
- gravação parcial da configuração: os campos não enviados mantêm o valor salvo (antes eram zerados);
- origem de cada parâmetro de conexão registrada no log, falha de credenciais explicada e mensagens de depuração removidas.

2.3 Medidor de vazão (calc.py, app.py, trials.py)

O método (analógico ou pulsos) é escolhido no card de Vazão e gravado no config.json. Não escrevemos nada no datalogger: com o programa v4 ele mede os dois canais em todo Scan, e o servidor calcula a vazão pelo canal escolhido. O método só aparece como ativo depois que uma amostra do datalogger é calculada com ele (Figura 4). Durante um ensaio a troca é recusada, e um ensaio de vazão não começa enquanto o método escolhido não estiver confirmado.

Figura 4 – Escolha e confirmação do método



Fonte: app.py (/api/flow/method) e static/js/flow.js.

No 4–20 mA, o servidor passou a gravar sempre o mA bruto e a classificar o sinal (Figura 5). Antes, corrente fora de 4–20 mA virava "leitura inválida" e o valor era descartado, e o limite de 4,000 mA era exato: a vazão zero só era aceita porque o medidor parado lia 4,011 mA. Dentro de uma tolerância configurável (0,02 mA por padrão), a escala fica presa no limite, sem extrapolar; abaixo de 3,6 mA o sinal é tratado como falha.

Figura 5 – Classificação do sinal 4–20 mA



Esquemático, fora de escala. Tolerância nos limites: 0,02 mA (padrão), configurável de 0 a 0,2 mA.

Fonte: calc.classify_ma; manual do MAG 5000/6000 [1]; NAMUR NE 43 [3].

Nos pulsos, os cálculos são feitos no servidor, a partir do dado bruto:

$$f = N / \Delta t \qquad Q = 60 \cdot f / K \qquad V = \Delta N_{acum} / K$$

em que N é a contagem do registro; Δt, o intervalo real até o registro anterior; K, o fator em pulsos/L; Q, a vazão em L/min; ΔN_{acum}, a variação do contador acumulado do datalogger durante o ensaio; e V, o volume. Como o volume usa o acumulado, um registro perdido não faz perder pulsos. O K pode ser informado em pulsos/L, pulsos/m³, L/pulso ou mL/pulso e é guardado em pulsos/L. Não há K padrão: o 0,01 pulso/L que já estava no config.json não tem origem registrada e precisa ser conferido no MAG 5000.

Nos ensaios, o método, o K e a tolerância ficam congelados no início. O resultado mostra a contagem, a resolução e o volume pelo outro canal, e avisa quando o ensaio tem menos de 200 pulsos.

2.4 Programa do datalogger (CRBasic v4)

Escrevemos o Pluviometria_v4_vazao_pulsos.CR1X a partir do programa em uso (v3). As rotinas da balança, do termopar e do pluviômetro são idênticas, e entra um único PulseCount em P2, em modo de alta frequência [2]. A tabela Vazao1s ganhou três campos (Tabela 2), e os quatro primeiros ficaram iguais, para o servidor anterior continuar lendo o programa novo. Corrigimos também a leitura NAN do 4–20 mA, que saía com Flow_Status = 0. O programa ainda não foi compilado: não havia CRBasic Editor na máquina.

Tabela 2 – Campos da tabela Vazao1s

Campo	Conteúdo	Programa
Flow_mA	corrente do medidor em RG1 (mA)	v3 e v4
Flow_Status	0 na faixa, 1 abaixo, 2 acima; 3 sem leitura (NAN)	v3 e v4 (3 só na v4)
Flow_mode	mantido em 2 para o servidor anterior continuar lendo a v4	v3 e v4
Flow_Pulsos	pulsos contados em P2 no Scan (contagem, não frequência)	v4
Flow_PulsosTot	soma das contagens válidas desde o início do programa	v4
Flow_PStatus	0 válido, 1 primeiro Scan, 2 saturado, 3 acima de 12 kHz, 4 sem leitura	v4

Fonte: Pluviometria_v3_com_vazao_4-20mA.CR1X e Pluviometria_v4_vazao_pulsos.CR1X.

No servidor, o perfil programa_vazao trata os campos de pulso como opcionais, então funciona com a v3 e com a v4. A ferramenta tools/probe_cr1000x.py passou a conferir os campos do medidor e a apontar grafia diferente, como Flow_Mode × Flow_mode.

2.5 Interface (templates/, static/)

Refizemos a interface nas cores da Elus. No Método Comparativo, o card do medidor fica ao lado da balança, com o seletor Analógico/Pulso, o estado de confirmação, o sinal bruto e a comparação entre os dois canais (Figura 6). Os parâmetros ficam num painel lateral com três abas, Pulsos (fator K), Analógico 4–20 mA e Diagnóstico (Figura 7). O painel valida cada campo ao sair dele e mostra a resolução e a vazão que a leitura atual teria com os valores digitados. As transições respeitam a opção do sistema de reduzir movimento, tudo funciona pelo teclado e a tela cabe em 400 px de largura.

Figura 6 – Card do medidor de vazão no Método Comparativo



Fonte: captura da interface com o emulador (dados sintéticos).

Figura 7 – Painel de parâmetros do medidor, aba Pulsos



Fonte: captura da interface com o emulador (dados sintéticos).

2.6 Testes

Tabela 3 – Resultados dos testes

Conjunto	Resultado	Observação
Testes automatizados, antes das alterações	73 aprovados, 3 reprovados	os 3 reprovados cobriam um programa que nunca foi carregado
Testes automatizados, depois das alterações	221 aprovados	medidor, adaptador, API, ensaios, emulador com os esquemas v3 e v4, equivalência da otimização
Ponta a ponta: ensaio	36 de 36	inclui troca de método e recusa durante ensaio
Ponta a ponta: programa atual	17 de 17	recuperação após queda, cursor e reinício
Ponta a ponta: perfil elus_calib	16 de 16	regressão do perfil anterior
Ponta a ponta: ensaio com leitura paralela	36 de 36	DATALOGGER_PARALLEL=1, contra o emulador
Respostas das rotas, código anterior × otimizado	69 de 69 idênticas	comparação byte a byte, incluindo os 3 ensaios do banco real
Interface no navegador	aprovada	Chrome, larguras de 1440 px e 400 px, emulador v4 e v3

Fonte: execução de pytest e dos roteiros tests/e2e_*.py em 28/10/2026.

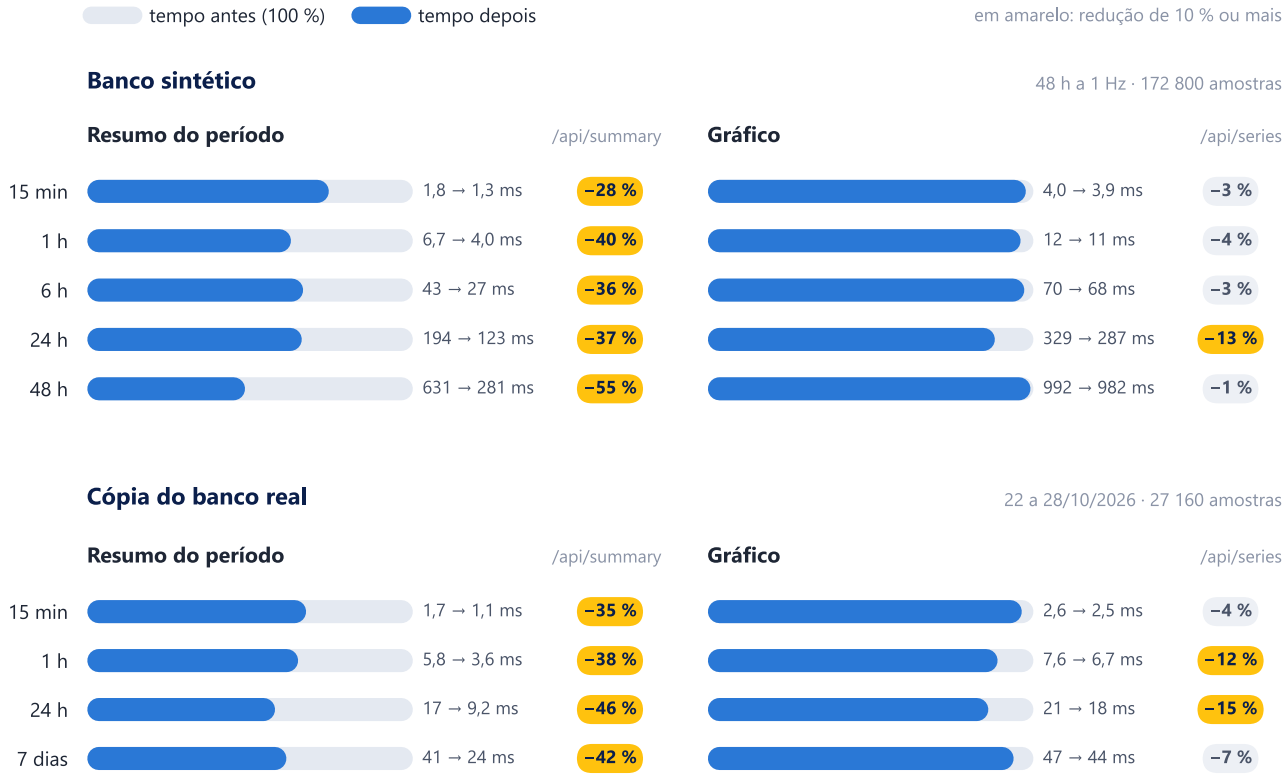
Todos os testes rodaram contra um emulador da API do CR1000Xe, com dados sintéticos e pastas temporárias; não mexemos no banco, no cursor nem na configuração reais. Eles cobrem os casos da seção 9 do pedido: no analógico, corrente perto de 4 e de 20 mA, intermediária, abaixo e acima da faixa, sem leitura e com faixa incompleta; nos pulsos, ausência de pulsos, um pulso, frequência estável e variável, acima da faixa, K válido, ausente e negativo, conversão de unidades, janela irregular, lacuna e acumulado reiniciado; na troca

de método, seleção, confirmação, recusa, reinício do servidor, persistência e ensaio em andamento; e a regressão de balança, termopar, pluviômetro, histórico, gráficos, API, configurações e ensaios.

2.7 Desempenho

Medimos antes de mexer: só adquirindo, o servidor usa 0,15 % de um núcleo de CPU, e na janela padrão do painel (15 min) as rotas respondem em poucos milissegundos. O custo estava nas janelas longas. Calculando a cobertura do período uma vez só (eram duas) e montando a configuração do medidor uma vez por requisição (era uma por linha), o resumo do período ficou de 28 % a 55 % mais rápido e o gráfico, até 15 % (Figura 8). Gravamos 69 respostas das rotas com o código anterior e comparamos com o novo: todas idênticas, byte a byte.

Figura 8 – Tempo de processamento no servidor, antes e depois da otimização



Fonte: medições de 28/10/2026 numa instância isolada, código anterior × otimizado, mediana de 9 chamadas em 3 rodadas alternadas.

3 Atendimento ao pedido

3.1 Item a item

A Tabela 4 relaciona cada requisito do pedido com o que fizemos; os números entre parênteses são as seções do texto original. Na tabela, “atendido” quer dizer implementado e verificado em emulador: nenhum item foi verificado no CR1000Xe real nem no MAG 5000.

Tabela 4 – Atendimento ao pedido

Requisito do pedido (item)	O que fizemos	Situação
Seletor Analógico/Pulso no card de Vazão; método ativo só depois de confirmado (2.1, 6.6)	Seletor no card. O método solicitado vai para o config.json e só aparece como ativo depois que uma amostra do datalogger é calculada com ele (seção 2.3)	Atendido
Troca de método bloqueada durante ensaio (2.1)	Troca recusada durante ensaio; ensaio de vazão não começa com método não confirmado; método e parâmetros congelados no resultado	Atendido

Requisito do pedido (item)	O que fizemos	Situação
Como a seleção chega ao datalogger, sem escrita não analisada (2, 4.1)	O CR1000Xe mede os dois canais em todo Scan e a escolha é feita no servidor. Nada é escrito no datalogger, que continua acessado com conta somente leitura	Atendido, sem escrita no datalogger
Preservar o 4–20 mA; limites de 4 e 20 mA sem extrapolação (2.2)	Mesmos parâmetros (vazão em 4 e 20 mA, unidade); mA bruto sempre gravado; tolerância nos limites sem extrapolar; falha abaixo de 3,6 mA (seção 2.3)	Atendido
Variáveis de pulso no CRBasic; contagem não tratada como frequência (2.3)	Flow_Pulsos, Flow_PulsosTot e Flow_PStatus (seção 2.4). A frequência é calculada no servidor como N / Δt, com o intervalo real entre registros	Implementado; programa v4 ainda não carregado
Baixas vazões e altas frequências (2.3)	Aviso quando a corrente indica vazão e não chegam pulsos; resolução por pulso exibida; aviso de ensaio com menos de 200 pulsos; entrada de alta frequência com limite de 12 kHz no programa	Atendido
Fator K no Método Comparativo, com unidade, validação e persistência (3, 3.1, 3.3)	Painel do medidor aberto a partir do Método Comparativo, com abas Pulsos, Analógico e Diagnóstico. K em pulsos/L, pulsos/m³, L/pulso ou mL/pulso, guardado em pulsos/L no config.json; validação na interface e no servidor	Atendido
Conversões e casos-limite (3.2)	Divisão por zero, K ausente ou negativo, frequência zero ou acima da faixa e janela irregular tratados com estado e motivo. Vazão em L/min, L/h, mL/min, mL/s, m³/h, m³/min ou m³/s	Atendido
Não inventar o fator K do MAG 5000 (3.2, 11)	Nenhum K padrão. O valor de 0,01 pulso/L já gravado foi mantido e marcado como sem origem	**Pendente: conferir no MAG 5000**
Revisão integral do CRBasic, sem contagem duplicada (4)	Programa v4 derivado do v3 em uso, com um único PulseCount em P2; leitura NAN do 4–20 mA corrigida	Implementado; compilação pendente
Ligação elétrica sem presumir compatibilidade (4.2)	Requisitos levantados no manual: terminais 56/57/58 e saída ativa de 24 V, acima do limite de ±20 V da entrada. Proposta com saída passiva e pull-up, marcada como não validada (docs/CRBASIC_V4_PULSOS.md)	**Pendente: ligação na bancada**
Revisão do Python e dos nomes de campos (5, 5.1)	Corrigidas a releitura indevida e as duas cópias do servidor (seções 2.1 e 2.2); o probe confere os campos de vazão e a grafia, como em Flow_mode	Atendido
Modelo de dados e estados de erro; erro nunca vira zero (5.2, 5.3)	Cada leitura traz método solicitado e ativo, valor bruto, valor convertido, unidade, estado e motivo. Comunicação, dados desatualizados, configuração, sinal e confirmação têm estados próprios	Atendido
Interface: redesenho, transições e acessibilidade (6)	Interface refeita nas cores da Elus; painel lateral com abas; validação ao sair do campo; transições discretas e redução de movimento; navegação por teclado; largura de 400 px sem rolagem horizontal	Atendido
Desempenho e ciclos de recuperação (7)	Os ciclos de recuperação eram a releitura indevida (seção 2.1). Agenda de consultas sem aumentar o número de consultas; resumo do período até 55 % mais rápido; mensagens de depuração removidas; log com rotação	Atendido; CPU medida (0,15 % de um núcleo); memória não medida
Integridade metrológica (8)	Nenhuma suavização nos valores usados em ensaio; médias só na exibição; implementação e verificação separadas da validação, que fica com o laboratório (seção 3.2)	Atendido
Testes obrigatórios (9)	Casos de 4–20 mA, pulsos, troca de método e regressão, com emulador e dados sintéticos isolados; arquivos reais não alterados (seção 2.6)	Atendido em emulador

Requisito do pedido (item)	O que fizemos	Situação
Método de trabalho e entrega (10)	Inventário, diagnóstico, plano e matriz de dependências no relatório de diagnóstico; pacote de entrega com instalação, atualização do CR1000Xe, testes, pendências e reversão (docs/ENTREGA_2026-10-28.md)	Atendido

Fonte: pedido (Anexo A), código entregue e resultados da seção 2.6.

3.2 O que depende da bancada

Todos os requisitos de software estão implementados. O que falta depende do equipamento:

- compilar e carregar o programa v4 no CR1000Xe e conferir os campos com tools/probe_cr1000x.py;
- conferir no MAG 5000 o fator K (volume por pulso, ou f_{max}/Q_{max}), o Q_{max} da faixa 4–20 mA, o corte de vazão baixa e a constante de tempo;
- ligar a saída de pulsos em modo passivo, com resistor de pull-up, depois de conferir os terminais na unidade instalada. Nunca ligar a saída ativa de 24 V em P2, que aceita no máximo ±20 V [2];
- fazer a validação experimental e a avaliação metrológica, que não dependem do software.

O passo a passo está em docs/CRBASIC_V4_PULSOS.md e docs/ENTREGA_2026-10-28.md, dentro do pacote de entrega.

Referências

[1] SIEMENS. *SITRANS F M MAGFLO MAG 5000/6000: operating manual*. Edição 05/2006.

[2] CAMPBELL SCIENTIFIC. *CR1000X: pulse measurement specifications* e *CRBasic Help: PulseCount, PConfig e POption*. Disponível em: help.campbellsci.com.

[3] NAMUR. *NE 43: standardization of the signal level for the failure information of digital transmitters*.

Apêndice A – Arquivos alterados

Arquivo	Alteração
crbasic/Pluviometria_v4_vazao_pulsos.CR1X	novo: canal de pulsos em P2, contador acumulado e estado; NAN no 4–20 mA
crbasic/Pluviometria_v3_com_vazao_4-20mA.CR1X	cópia do programa em uso, incluída no projeto
profiles/programa_vazao.json	campos de pulso opcionais; compatível com v3 e v4
datalogger.py	registro âncora, detecção de reinício e lacuna, dado bruto do medidor preservado; leitura paralela opcional (desligada)
calc.py	funções do medidor: fator K, classificação do 4–20 mA, janela real, volume por acumulado; cobertura do período calculada uma vez
app.py	instância única, agenda das consultas, gravação sem regressão, API do medidor, configuração parcial; configuração do medidor montada uma vez por requisição
trials.py	método e parâmetros congelados no ensaio; volume pelo acumulado; outro canal
templates/, static/	interface refeita; card do medidor; painel de parâmetros; gráficos ao vivo
tools/probe_cr1000x.py	verifica os campos do medidor de vazão e a grafia dos nomes
tests/	emulador com os esquemas reais; testes do medidor, da API, do adaptador e da equivalência da otimização
docs/	ENTREGA_2026-10-28.md, CRBASIC_V4_PULSOS.md e DESEMPENHO_BACKEND.md novos; README e mapa de variáveis atualizados

Pacote de entrega: _entrega_2026-10-28/monitor-pluviometro_entrega_2026-10-28.zip (código e documentação; sem banco, cursor, configuração nem log).

Anexo A – Pedido original (texto integral)

Texto do pedido do responsável técnico, reproduzido sem alteração de conteúdo; só a formatação foi adaptada para o relatório.

PROJETO ELUS — REVISÃO COMPLETA, OTIMIZAÇÃO E IMPLEMENTAÇÃO DA AQUISIÇÃO DE VAZÃO POR PULSOS NO CR1000Xe

Quero que você utilize **o máximo das suas capacidades de engenharia de software, instrumentação, sistemas embarcados, metrologia, Python, CRBasic, arquitetura de sistemas e Front-end Design** para analisar, corrigir, otimizar e evoluir o projeto de monitoramento e calibração de pluviômetro e medidor de vazão da Elus Instrumentação.

Estou enviando dois conjuntos de arquivos:

1. **Programa CRBasic do datalogger Campbell Scientific CR1000Xe**, responsável pela aquisição dos instrumentos.
2. **Projeto completo do software do pluviômetro**, incluindo Python, Flask, Socket.IO, HTML, CSS, JavaScript, configurações, perfis de aquisição, persistência e demais arquivos existentes.

Não quero uma análise superficial. Quero uma revisão técnica profunda, seguida de uma implementação coerente, funcional, integrada e visualmente refinada.

1. Contexto atual do projeto

O sistema utiliza um datalogger Campbell Scientific CR1000Xe para adquirir informações de instrumentos de laboratório.

Atualmente, o sistema já possui aquisição de:

- Balança de precisão MK30K1.
- Termopar tipo T para temperatura da água.
- Pluviômetro de báscula.
- Medidor de vazão eletromagnético Siemens SITRANS FM MAG 5000, com aquisição analógica de 4–20 mA já integrada ao software.

A comunicação entre o CR1000Xe e o servidor Python está funcionando. O servidor consulta as tabelas do datalogger, interpreta as leituras, calcula os valores necessários e os disponibiliza para a interface.

O projeto utiliza, entre outros componentes:

- CRBasic no datalogger.
- Python no servidor.
- Flask e Socket.IO na aplicação web.
- Perfis de aquisição e arquivos de configuração.
- Tabelas do CR1000Xe, incluindo `Vazao1s`.
- Um perfil de aquisição chamado `programa_vazao`.

O perfil atualmente utilizado contém a seguinte estrutura lógica:

- Tabela: `Vazao1s`.
- Modo: `Flow_mode`.
- Corrente: `Flow_mA`.
- Estado: `Flow_Status`.

Importante: o campo real retornado pelo datalogger é `Flow_mode`, com `m` minúsculo. Essa diferença já foi corrigida no perfil Python.

O programa CRBasic registra valores como `Flow_mV`, `Flow_mA`, `Flow_Status` e `Flow_mode`. O modo analógico já foi validado no servidor, e as leituras estão chegando corretamente.

O modo atual utiliza a corrente de 4–20 mA e os parâmetros de vazão correspondentes a 4 mA e 20 mA.

Não quero perder essa funcionalidade durante a implementação dos pulsos.

2. Objetivo principal: implementar a aquisição por pulsos

Quero adicionar a possibilidade de adquirir a vazão por meio da saída digital de pulsos do Siemens SITRANS FM MAG 5000.

A aplicação deve permitir alternar entre dois métodos de aquisição:

1. **Analógico — 4–20 mA.**
2. **Pulsos — contagem/frequência de pulsos.**

Essa seleção deve ser realizada diretamente na interface, no card de Vazão, por meio de um controle simples, intuitivo e visualmente elegante, semelhante ao botão ou seletor de estado atualmente utilizado para a função `Off`.

Não quero que o usuário precise editar arquivos JSON, modificar o código Python ou alterar manualmente o CRBasic para alternar o método de aquisição durante o uso normal da interface.

Entretanto, não presuma que uma alteração no seletor visual seja suficiente para mudar fisicamente o modo de aquisição. Analise como a seleção precisa ser propagada de forma segura entre a interface, o servidor e o datalogger.

2.1. Comportamento esperado do seletor

No card de Vazão, crie um controle compacto que permita escolher entre:

- `Analógico`
- `Pulso`

A seleção ativa deve ficar visualmente evidente, sem ocupar espaço excessivo.

O controle deve:

- Ter aparência integrada ao restante do card.
- Ser fácil de compreender à primeira vista.
- Utilizar estados visuais claros.
- Possuir transições suaves.
- Indicar qual método está efetivamente ativo no sistema.
- Evitar mostrar um método como ativo quando a aquisição real ainda estiver utilizando outro.
- Tratar falhas de comunicação, indisponibilidade do método ou divergências entre o método solicitado e o método efetivamente utilizado.
- Impedir alterações acidentais durante um ensaio de calibração, se isso puder comprometer a integridade dos dados.

Se necessário, diferencie visualmente o método selecionado pelo usuário do método confirmado pelo CR1000Xe.

2.2. Comportamento do modo analógico

Preserve integralmente a funcionalidade existente de 4–20 mA.

O modo analógico deve continuar utilizando os parâmetros já implementados:

- Vazão correspondente a 4 mA.
- Vazão correspondente a 20 mA.
- Unidade de vazão.
- Validação da corrente.
- Tratamento de valores fora da faixa.

- Estados de leitura inválida, ausência de comunicação e configuração incompleta.

Não substitua nem simplifique a lógica existente sem entender suas dependências.

Verifique especialmente o tratamento de valores próximos aos limites de 4 mA e 20 mA, sem introduzir extrapolações indevidas.

2.3. Comportamento do modo por pulsos

Quando o usuário selecionar `Pulso`, a aplicação deve passar a utilizar a aquisição digital correspondente, desde que essa aquisição esteja devidamente configurada e confirmada.

Analise quais variáveis precisam ser disponibilizadas pelo CRBasic. Considere, conforme a estratégia tecnicamente adequada:

- Quantidade de pulsos por intervalo de aquisição.
- Frequência de pulsos em Hz.
- Estado da entrada digital.
- Estado de validade da leitura.
- Indicadores de saturação, perda de dados ou frequência incompatível.
- Timestamp ou referência temporal para os cálculos.

Não adicione variáveis redundantes sem justificativa.

A implementação deve preservar a resolução temporal necessária para calcular corretamente a vazão.

Não trate a quantidade de pulsos acumulada como se fosse automaticamente uma frequência em Hz. Defina claramente a janela de contagem e o intervalo de tempo utilizado no cálculo.

Considere o comportamento em baixas vazões, nas quais podem ocorrer intervalos sem pulsos, e em altas frequências, nas quais a aquisição deve evitar perder eventos.

Se a frequência do sinal ou o tipo de saída do MAG 5000 exigir uma configuração específica do CR1000Xe, identifique essa dependência antes de concluir a implementação.

3. Configurações do fator K de pulsos

Quero que as configurações do fator K apareçam dentro do contexto do **Método Comparativo**.

Ao clicar em `Método Comparativo`, a interface deve disponibilizar uma aba, seção ou painel específico para configurar os parâmetros da aquisição por pulsos.

Essa área deve ser organizada de forma clara, sem misturar desnecessariamente os parâmetros de calibração do pluviômetro com os parâmetros do medidor de vazão.

3.1. Parâmetros necessários

Implemente os campos necessários para configurar o fator K do medidor.

Considere pelo menos:

- Fator K.
- Unidade de referência do fator K.
- Unidade de vazão desejada na interface.
- Parâmetros adicionais somente quando forem necessários para a conversão ou para a validação da aquisição.

O fator K pode ser expresso, por exemplo, em pulsos/L ou pulsos/m³. A implementação deve tratar corretamente as unidades e evitar ambiguidades.

Se for mais seguro e consistente internamente, normalize o cálculo para uma unidade de referência e converta a vazão somente na apresentação.

Não presuma que todos os medidores utilizam o mesmo fator K.

3.2. Conversão de pulsos para vazão

Implemente a conversão com base no fator K e no intervalo de aquisição efetivamente utilizado.

Por exemplo, se o fator K estiver expresso em pulsos/L e a frequência for medida em Hz:

$$Q_{L/min} = f_{Hz} / K_{pulsos/L} \times 60$$

Essa equação é apenas um exemplo. A implementação final deve tratar corretamente a unidade selecionada e documentar as conversões.

Se a aquisição trabalhar com quantidade de pulsos em uma janela temporal, utilize a quantidade contada e a duração real dessa janela.

Verifique:

- Conversão entre pulsos/L e pulsos/m³.
- Conversão entre L/min, L/h e m³/h, conforme as unidades suportadas.
- Divisão por zero.
- Fator K ausente, nulo ou negativo.
- Frequência zero.
- Frequência fora da faixa tecnicamente admissível.
- Leituras inválidas ou incompletas.
- Mudanças de configuração durante um ensaio.
- Arredondamentos e precisão numérica.

Não invente um fator K para o MAG 5000. Utilize apenas um valor configurado pelo usuário ou documentado em uma fonte técnica confiável.

3.3. Persistência e validação

As configurações devem ser persistidas no mecanismo de configuração já utilizado pelo projeto, respeitando sua arquitetura atual.

Verifique:

- Validação no Front-end.
- Validação no Back-end.
- Persistência após reiniciar o servidor.
- Valores padrão seguros.
- Compatibilidade com configurações anteriores.
- Tratamento de configurações ausentes ou inválidas.
- Consistência entre a configuração salva e o método realmente utilizado.

Não crie uma segunda fonte de verdade desnecessária.

Se a configuração atual já possuir um mecanismo adequado, reutilize-o.

4. Revisão completa do programa CRBasic

Quero que você analise o arquivo CRBasic integralmente antes de modificar qualquer parte.

Não faça alterações isoladas sem compreender a estrutura geral do programa.

Analise:

- Declarações de variáveis.
- Configuração das entradas analógicas e digitais.
- Instruções de aquisição.
- Temporização e sincronização.
- Instruções `PulseCount()` ou equivalentes.
- Tabelas de armazenamento.
- Campos registrados em `Vazao1s`.

- Modos de operação.
- Flags de estado.
- Cálculos de frequência e vazão.
- Tratamento de falhas.
- Consistência entre nomes de variáveis e nomes de campos das tabelas.
- Possíveis conflitos entre tarefas de aquisição.
- Uso de recursos do CR1000Xe.
- Compatibilidade com a versão do compilador utilizada.

Determine se o programa atual já possui uma implementação parcial da aquisição por pulsos.

Se existir, aproveite-a quando estiver correta. Se estiver incompleta, corrija-a. Se precisar ser reestruturada, explique por quê.

Não duplique a contagem de pulsos nem crie duas rotinas concorrentes para a mesma entrada.

4.1. Arquitetura de aquisição

Quero que o programa seja organizado de forma clara, mantendo os outros instrumentos funcionando.

A aquisição por pulsos deve coexistir com a aquisição analógica sem interferir na balança, no termopar ou no pluviômetro.

Defina claramente como o modo de aquisição é selecionado e como essa seleção chega ao CR1000Xe.

Se o datalogger precisar receber uma configuração por comando, variável de controle ou mecanismo já existente, implemente isso somente após verificar a viabilidade e os riscos.

Não habilite operações de escrita no datalogger sem analisar os mecanismos de comunicação, segurança e persistência utilizados pelo projeto.

4.2. Entradas e segurança elétrica

Não invente a pinagem do MAG 5000 nem presuma que sua saída digital seja compatível com a entrada escolhida.

O tipo de saída pode depender da configuração elétrica e funcional do transmissor.

Identifique as informações necessárias para confirmar a ligação, incluindo:

- Tipo de saída.
- Terminais utilizados.
- Modo ativo ou passivo, quando aplicável.
- Necessidade de alimentação ou resistor externo.
- Faixa de tensão.
- Tipo de entrada do CR1000Xe.
- Frequência máxima esperada.

Se não houver informação suficiente nos arquivos enviados, identifique explicitamente o que precisa ser confirmado no manual ou no equipamento.

Não apresente uma ligação elétrica hipotética como se estivesse validada.

5. Revisão do Python e da comunicação com o datalogger

Revise integralmente o fluxo de dados entre o CR1000Xe e a aplicação.

Quero entender e corrigir possíveis problemas em:

- `datalogger.py`.
- `app.py`.
- `calc.py`.
- Perfis de aquisição.
- Arquivos JSON.
- APIs.
- Rotinas de consulta das tabelas.
- Atualização da interface.
- Persistência das configurações.
- Cálculos e estados do medidor.
- Logs e tratamento de exceções.

Identifique todas as dependências entre o programa CRBasic e o servidor Python.

5.1. Campos e nomes

Confira cuidadosamente a correspondência entre os campos registrados pelo datalogger e os nomes esperados pelo Python.

Já houve um problema causado pela diferença entre:

- `Flow_Mode`
- `Flow_mode`

O nome correto do campo retornado pela tabela atual é `Flow_mode`.

Não repita esse erro. Verifique todos os campos e seus nomes reais antes de modificar o código.

5.2. Modelo de dados

Defina uma estrutura consistente para representar a leitura do medidor independentemente do método de aquisição.

O modelo deve distinguir claramente:

- Método solicitado.
- Método efetivamente ativo.
- Valor bruto adquirido.
- Valor convertido.
- Unidade.
- Estado da leitura.
- Timestamp.
- Motivo de uma eventual leitura inválida.

Evite misturar os estados de comunicação, aquisição e configuração.

Uma leitura pode estar chegando corretamente do datalogger e, mesmo assim, ser inválida para o cálculo da vazão. O sistema deve representar essas situações de maneira diferente.

5.3. Estados de erro

Revise os estados utilizados pela interface e pelo Back-end.

Considere estados distintos para:

- Comunicação indisponível.
- Dados desatualizados.
- Método não configurado.
- Leitura inválida.
- Corrente fora da faixa.
- Configuração incompleta.
- Falha de aquisição de pulsos.
- Método selecionado ainda não confirmado pelo datalogger.

Não transforme silenciosamente erros em valores zero.

Não apresente dados simulados, fictícios ou antigos como se fossem leituras atuais.

6. Redesign completo da interface — Front-end Design

Esta parte é muito importante.

Quero que você utilize intensamente suas capacidades de **Front-end Design, UX/UI, design de produto, hierarquia visual, animação e composição de interfaces profissionais**.

A interface deve parecer um sistema de instrumentação desenvolvido profissionalmente para um laboratório de metrologia, e não um painel genérico cheio de cartões e informações concorrentes.

Mantenha a identidade visual da Elus, respeitando o estilo existente, especialmente os tons de azul-escuro e amarelo, mas refine a composição, a hierarquia, os espaçamentos e os detalhes visuais.

Não quero apenas trocar cores ou arredondar cartões. Quero uma evolução visual coerente.

6.1. Direção visual

Busque uma interface:

- Elegante.
- Moderna.
- Técnica.
- Minimalista, sem perder informações importantes.
- Visualmente equilibrada.
- Consistente em toda a aplicação.
- Fácil de compreender durante uma calibração.
- Adaptável a diferentes resoluções.
- Adequada para uso prolongado em ambiente de laboratório.

Use uma hierarquia tipográfica bem definida.

Evite excesso de bordas, badges, cores de alerta, sombras fortes, divisórias desnecessárias e textos redundantes.

Cada elemento deve ter uma função clara.

6.2. Cards de instrumentação

Revise o card de Vazão e os demais cards existentes.

O card de Vazão deve apresentar, de forma clara:

- Identificação do medidor.
- Método de aquisição ativo.
- Valor instantâneo da vazão.
- Unidade.
- Estado da aquisição.
- Acesso à configuração, quando apropriado.

O controle Analógico/Pulso deve ser compacto e integrado ao card, sem parecer um elemento improvisado.

O card deve mudar de maneira elegante quando o usuário alternar o método, mas sem esconder informações importantes ou confundir a leitura.

Evite exibir simultaneamente todos os parâmetros técnicos na tela principal. Coloque configurações detalhadas em um painel contextual.

6.3. Método Comparativo

Reorganize a área de Método Comparativo para que as configurações relacionadas à aquisição e aos parâmetros do ensaio sejam fáceis de encontrar.

A configuração do fator K deve aparecer em uma aba ou seção contextual, com uma apresentação visual coerente com o restante do sistema.

Se fizer sentido para a arquitetura existente, utilize:

- Abas compactas.
- Painéis expansíveis.
- Campos agrupados por finalidade.
- Ajuda contextual discreta.
- Unidades visíveis junto aos campos.
- Validação inline.
- Mensagens de erro claras.

Não introduza abas apenas por estética. A navegação deve realmente melhorar a organização.

6.4. Transições e microinterações

Utilize transições suaves e naturais para:

- Alternância entre Analógico e Pulso.
- Abertura e fechamento de painéis.
- Mudanças de estado.
- Atualização de valores.
- Exibição de mensagens de validação.
- Carregamento de configurações.
- Navegação entre abas.

As animações devem ser discretas e funcionais.

Evite movimentos exagerados, efeitos chamativos ou transições que prejudiquem a leitura dos dados em tempo real.

Respeite a preferência do sistema por redução de movimento, quando aplicável.

6.5. Consistência e acessibilidade

Verifique:

- Contraste.
- Legibilidade.
- Estados de foco.
- Navegação por teclado.
- Rótulos acessíveis.
- Mensagens de erro compreensíveis.
- Tamanho adequado dos controles.
- Comportamento em telas menores.

Não sacrifique a usabilidade técnica em favor da estética.

6.6. Integridade visual dos dados

A interface não deve sugerir que uma leitura é válida quando não existe confirmação do servidor ou do datalogger.

Se o usuário solicitar a mudança para pulsos, mas o CR1000Xe ainda estiver configurado para analógico, mostre isso explicitamente.

Não apresente uma animação de sucesso antes de confirmar que a mudança foi aplicada.

As atualizações de valores em tempo real devem ser suaves, mas não devem atrasar nem alterar os valores reais recebidos.

7. Desempenho e otimização

Analise o sistema como um todo, procurando redundâncias e gargalos.

Investigue:

- Frequência de consulta ao CR1000Xe.
- Latência da API.
- Número de consultas repetidas.
- Uso de CPU e memória.
- Consultas redundantes às tabelas.
- Processamento desnecessário no Front-end.
- Atualizações excessivas do DOM.
- Re-renderizações.
- Acúmulo de listeners.
- Gestão de conexões Socket.IO.
- Persistência desnecessária.
- Escritas excessivas no datalogger.
- Rotação e volume de logs.
- Sincronização temporal.
- Tratamento de lacunas nos dados.

Já ocorreram situações em que o servidor apresentou ciclos de recuperação com muitas leituras processadas em pouco tempo. Investigue se isso é esperado ou se pode indicar atraso, recuperação de consultas ou processamento repetido.

Não aumente indiscriminadamente a frequência de consulta apenas para tornar a interface aparentemente mais rápida.

A prioridade é preservar a aquisição correta e melhorar a responsividade sem sobrecarregar o datalogger.

Remova logs de depuração temporários depois de concluir a investigação, mantendo registros operacionais úteis.

8. Integridade metrológica

Este sistema será utilizado em contexto de laboratório. Portanto, a correção metrológica é mais importante do que a aparência.

Verifique cuidadosamente:

- Unidades de engenharia.
- Conversões.
- Resolução temporal.
- Intervalos de aquisição.
- Frequência real dos pulsos.
- Fator K.
- Corrente analógica.
- Limites válidos.
- Arredondamentos.
- Precisão numérica.
- Timestamps.
- Dados ausentes.
- Lacunas de aquisição.
- Estados de erro.
- Consistência entre valor bruto e valor calculado.

Não introduza filtros, médias ou suavizações que alterem silenciosamente os valores utilizados em ensaios.

Se uma filtragem visual for útil, mantenha-a separada dos dados metrológicos originais.

Não afirme que o sistema está validado metrologicamente apenas porque o código funciona.

Diferencie claramente:

1. Implementação de software.
2. Validação da aquisição elétrica.
3. Verificação dos cálculos.
4. Validação experimental.
5. Avaliação metrológica.

9. Testes obrigatórios

Antes de considerar a implementação concluída, crie ou execute testes adequados ao projeto.

Os testes devem verificar, quando tecnicamente aplicável:

Aquisição analógica

- Corrente próxima de 4 mA.
- Corrente intermediária.
- Corrente próxima de 20 mA.
- Corrente abaixo da faixa.
- Corrente acima da faixa.
- Ausência de leitura.
- Configuração incompleta.

Aquisição por pulsos

- Ausência de pulsos.
- Um ou poucos pulsos em uma janela.
- Frequência estável.
- Frequência variável.
- Frequência zero.
- Frequência acima da faixa suportada.
- Fator K válido.
- Fator K ausente ou inválido.
- Conversão entre unidades.
- Intervalos de aquisição irregulares.
- Perda ou atraso de dados.

Alternância entre métodos

- Seleção de Analógico.
- Seleção de Pulso.
- Confirmação do método ativo.
- Falha na mudança de modo.
- Reinício do servidor.
- Persistência da configuração.
- Recuperação da comunicação.
- Comportamento durante um ensaio ativo.

Regressão

Confirme que a implementação não prejudicou:

- Balança.
- Termopar.
- Pluviômetro.
- Histórico.
- Gráficos.
- API.
- Configurações existentes.
- Fluxo de calibração.

Não utilize dados fictícios para simular uma validação real. Testes automatizados com dados sintéticos são permitidos quando estiverem claramente isolados dos dados de produção e identificados como testes.

Não modifique o sistema operacional ou os arquivos de configuração reais para simular uma aquisição válida.

10. Método de trabalho obrigatório

Quero que você trabalhe de forma organizada, profunda e criteriosa.

Etapa 1 — Inventário

Examine todos os arquivos enviados.

Identifique a arquitetura atual, os pontos de entrada, as dependências, as tabelas, os perfis e os caminhos dos dados.

Etapa 2 — Diagnóstico

Antes de modificar o código, apresente:

- Problemas encontrados.
- Inconsistências.
- Riscos.
- Redundâncias.
- Limitações.
- Dependências entre arquivos.
- Pontos que precisam de confirmação física ou documental.

Classifique os problemas por gravidade e impacto.

Etapa 3 — Plano de implementação

Apresente um plano de alterações que identifique quais arquivos serão modificados e por quê.

Inclua uma matriz de dependências entre:

- Interface.
- API.

- Configuração.
- Python.
- Perfil de aquisição.
- CRBasic.
- Datalogger.
- Instrumento físico.

Etapa 4 — Implementação

Implemente a solução de forma integrada, evitando remendos isolados.

Preserve o que já funciona.

Não crie mecanismos paralelos quando a arquitetura existente puder ser estendida corretamente.

Se precisar fazer alterações estruturais, explique a necessidade.

Etapa 5 — Verificação

Execute os testes possíveis, revise a consistência do código e verifique se os arquivos estão completos.

Não declare que algo foi testado se não tiver sido efetivamente executado.

Etapa 6 — Entrega

Entregue os arquivos completos modificados, preferencialmente organizados em um pacote que possa substituir o projeto anterior após backup.

Inclua:

- Lista de arquivos modificados.
- Resumo das mudanças.
- Instruções de instalação.
- Instruções para atualizar o programa do CR1000Xe.
- Configurações que precisam ser preenchidas.
- Testes realizados e resultados.
- Pendências que dependam do equipamento físico.
- Procedimento de reversão caso algo não funcione.

11. Restrições importantes

1. Não invente dados reais do medidor.
2. Não invente o fator K do Siemens MAG 5000.
3. Não presuma que a saída de pulsos já esteja configurada.
4. Não invente pinagem nem compatibilidade elétrica.
5. Não remova a aquisição analógica funcional.
6. Não altere os cálculos metrológicos sem justificativa.
7. Não introduza dependências desnecessárias.
8. Não remova funcionalidades existentes sem explicar o motivo.
9. Não esconda erros de aquisição apresentando zero.
10. Não considere a implementação concluída sem verificar a integração entre as camadas.
11. Não faça apenas recomendações: implemente as alterações possíveis nos arquivos enviados.
12. Se algo depender de informação que não está disponível, identifique a lacuna e continue com as partes que podem ser executadas com segurança.
13. Não deixe blocos de código incompletos, pseudocódigo ou funções de exemplo no lugar da implementação final.
14. Não altere o funcionamento do sistema de calibração sem verificar as consequências.
15. Não utilize a interface para mascarar falhas de comunicação ou de configuração.
16. Não faça alterações destrutivas sem backup e sem explicar como reverter.

12. Resultado esperado

Quero um sistema profissional em que:

- A aquisição analógica de 4–20 mA continue funcionando.
- A aquisição por pulsos possa ser selecionada na interface.
- O fator K seja configurado no contexto do Método Comparativo.
- A conversão de pulsos para vazão seja matematicamente consistente.
- O método realmente ativo seja identificado com clareza.
- A configuração seja persistente e validada.
- O CR1000Xe e o Python permaneçam sincronizados.
- Os erros sejam tratados explicitamente.
- A interface tenha excelente acabamento visual, com hierarquia, transições suaves e microinterações discretas.
- O código seja organizado, documentado e sustentável.
- A integração seja verificável, sem depender de dados fictícios.

Quero que você trate esse trabalho como uma revisão de engenharia e uma evolução real de produto, não como uma simples adição de campos na interface.

Comece pelo inventário e diagnóstico dos arquivos enviados. Depois apresente o plano de implementação e prossiga com as alterações, sempre preservando a funcionalidade existente e identificando claramente qualquer etapa que exija validação física no equipamento.